

编译原理·词法分析（二）：有穷自动机 NFA 与 DFA

Lexical Analysis: NFA & DFA ——详细学习笔记

基于课件 2.Lexical Analysis_NFA_DFA.pdf（中山大学·赵帅）整理

本笔记说明

本笔记完整覆盖第 2 讲《词法分析：NFA 与 DFA》共 80 页。它承接第 1 讲——上一讲学会了用正则表达式（Regular Expression, RE）描述单词的模式（这是”规范”），本讲解决如何把规范变成能真正运行的识别器（这是”实现”）。主线是一条清晰的”流水线”：

RE $\xrightarrow{\text{Thompson}}$ NFA $\xrightarrow{\text{子集构造}}$ DFA $\xrightarrow{\text{最小化}}$ 最小 DFA $\rightarrow$ 表驱动实现 $\rightarrow$ 词法分析器。

每个知识点用 [p 页码] 标注来源；专业术语首次出现给出中文（English）对照与通俗解释。

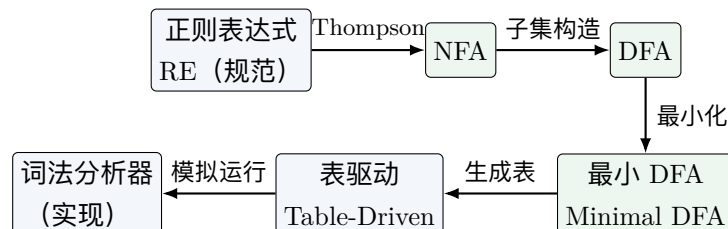
目录

1 全局蓝图：从规范到实现（第 1–2 页）	1
2 有穷自动机基础（第 3–13 页）	2
2.1 为什么需要有穷自动机 [p3,4]	2
2.2 转换图：FA 的画法 [p5]	2
2.3 FA 就是”语言识别程序” [p6,7]	3
2.4 DFA 与 NFA 的区别 [p8]	3
2.5 FA 的形式定义：五元组 [p9]	3
2.6 NFA 与 DFA 接受同一串的对比 [p10,11]	4
2.7 转换表：FA 的另一种表示 [p12,13]	4
3 Thompson 算法：RE $\rightarrow$ NFA（第 14–22 页）	4
3.1 基础：原子 RE 的 NFA [p14]	4
3.2 归纳：三条复合规则 [p15]	5
3.3 回顾：规范与实现如何衔接 [p18–22]	5
4 子集构造法：NFA $\rightarrow$ DFA（第 22–31 页）	6
4.1 为什么要转成 DFA [p22,23]	6
4.2 核心思想与两个待解决问题 [p24,25]	6
4.3 两个关键运算： ϵ -closure 与 move [p25,26]	6
4.4 完整算法（三步） [p27–30]	7

1 全局蓝图：从规范到实现（第 1–2 页）	2
5 DFA 最小化：分割法（第 32–40 页）	7
5.1 为什么能最小化、目标是什么 [p33]	7
5.2 分割算法 (Partition Algorithm) [p34]	8
6 复杂度分析（第 41–42 页）	9
7 表驱动实现与 Lex（第 43–62 页）	9
7.1 Lex 的流水线 [p50]	9
7.2 多模式合并成一个 NFA [p52]	10
7.3 模拟运行与”备份” [p53,54]	10
7.4 该匹配多长？消歧规则 [p55]	10
7.5 关键字怎么匹配 [p56]	10
7.6 Lex DFA 例子与二义性 [p57–62]	11
8 正则语言的极限与展望（第 63–68 页）	11
8.1 全讲转换流程小结 [p66,67]	11
9 练习与典型题（第 69–80 页）	12
考点速记	12

1 全局蓝图：从规范到实现（第 1–2 页）

本讲处在”从规范到实现 (From Specification to Implementation)”这张大地图的右半部分 [p1,2]。请先记住这条主链路——后面每一节都是在讲它的某一段：



四步走（本讲的四个核心算法） [p13]

RE → NFA：Thompson 算法 (Thompson Algorithm)。 **NFA → DFA**：子集构造法 (Subset-Construction Algorithm)。

DFA 最小化：分割法 (Partition Algorithm)。 **DFA → 表驱动实现** (Table-Driven Implementation)。

2 有穷自动机基础 (第 3–13 页)

2.1 为什么需要有穷自动机 [p3,4]

有穷自动机 (Finite Automata, FA) [p3,4]

RE 只是语言的”规范 (specification)”——它只定义了”哪些串属于这个语言”，本身不会”跑”。要真正识别串，需要一个实现：有穷自动机。

自动机 (automaton, 复数 automata) = 一台机器/一段程序；有穷自动机 = 状态数量有限的程序。它很像转换图 (transition diagram)：有状态和带标号的边，有唯一的开始状态、一个或多个接受状态。

一句话对应关系 [p4]

正则表达式 = 规范 (要识别什么)；有穷自动机 = 实现 (怎么识别)。两类自动机：非确定有穷自动机 (NFA, Nondeterministic Finite Automata) 与确定有穷自动机 (DFA, Deterministic Finite Automata)。

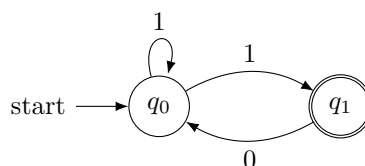
2.2 转换图：FA 的画法 [p5]

转换图的两类要素 [p5]

节点 (Node) = 状态 (state)：每个状态代表识别过程中”可能出现的一种情形”。

- 初始态 (Start)：唯一，用一个标着 start 的箭头指向它。
- 接受态 / 终态 (Final / Accepting)：可有多，用双圈表示。

边 (Edge) = 转移 (transition)：有向，标注输入符号；表示”读到该符号时从一个状态走到另一个状态”。



单圈 = 普通态，双圈 = 接受态，start 箭头指开始态。

2.3 FA 就是”语言识别程序” [p6,7]

FA 的语言 (Language of FA) [p6]

FA 是一个给字符串分类的程序, 输出**接受 (accept)**或**拒绝 (reject)**, 也就是识别一门语言的程序。

对给定串 x : 若存在一条转移序列把它从**开始态**带到某个**接受态**, 则称 x **被该 FA 接受**, 否则**拒绝**。

$L(\text{FA}) =$ **被该 FA 接受的所有串的集合**; 并且有 $L(\text{FA}) \equiv L(\text{RE})$ ——FA 与 RE 描述能力相同。

小例子 [p7]

上图 ($\Sigma = \{0, 1\}$) 识别的语言是”任意多个 1 之后跟恰好一个 0”。判定: $0\checkmark$ 、 $1\times$ 、 $11110\checkmark$ 、 $11101\times$ 、 $11100\times$ 、 $1111110\checkmark$ 。

2.4 DFA 与 NFA 的区别 [p8]

DFA vs. NFA [p8]

确定有穷自动机 (DFA): 任一时刻只处于一个状态。

- 每个状态对每个输入符号恰有一个转移; 没有 ϵ 转移 (no ϵ -moves); 在状态图中只走一条路径。

非确定有穷自动机 (NFA): 可同时处于多个状态。

- 一个状态对同一输入可有多个转移; 允许 ϵ 转移 (不读输入就能跳转); 可”选择”走哪条路。
 - **接受规则**: 只要存在某一条路径在输入结束时停在接受态, NFA 就**接受**。
-

NFA 的”非确定”怎么理解

把 NFA 想成”可以同时分身走所有岔路”: 只要有任意一个分身最终落在接受态, 整串就算被接受。而 DFA 像”一条道走到黑”: 每一步路线唯一, 毫无悬念。

2.5 FA 的形式定义: 五元组 [p9]

FA 的五元组 $(\Sigma, S, n, F, \delta)$ [p9]

- Σ : **输入字母表** (input alphabet)。

- S : **状态集** (set of states)。
- $n \in S$: **开始态** (start state)。
- $F \subseteq S$: **接受态集** (accepting states)。
- δ : **转移函数** (transitions), 形如 $\delta: S \times \text{Input} \rightarrow S$ (NFA 中结果是状态集合)。

2.6 NFA 与 DFA 接受同一串的对比 [p10,11]

以 $(a|b)^*abb$ 、输入 aabb 为例:

- **NFA** [p10]: 有多条可能走法。只需找到一条成功序列即可接受: $0 \xrightarrow{a} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{b} 3$ (终点 3 是接受态)。也存在失败序列 (一直停在 0), 但有一条成功即接受。
- **DFA** [p11]: 走法唯一: $0 \xrightarrow{a} 1 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{b} 3$, 要么到接受态、要么被拒。

2.7 转换表: FA 的另一种表示 [p12,13]

转换表 (Transition Table) [p12]

把 FA 的转移写成一张表: 行 = 状态, 列 = 输入符号, 格子 = 转移到的状态。

- **优点**: 能快速查到“某状态在某输入下转去哪”。
- **缺点**: 占空间——字母表大时, 多数状态对多数符号都没有转移, 表却仍要留位。
- **空间需求**: $O(|S| \times |\Sigma|)$ (状态数 $\times$ 字母表大小)。

3 Thompson 算法: $RE \rightarrow NFA$ (第 14–22 页)

Thompson 构造法 (Thompson Algorithm) [p14,15]

一种把任意正则表达式系统地转换为 NFA 的方法。核心思想是递归: 先给原子 RE 造小 NFA, 再用三条归纳规则把小 NFA 像积木一样**拼接**成大 NFA。拼出的 NFA 总是恰有一个开始态、一个接受态。

3.1 基础: 原子 RE 的 NFA [p14]

- ε 的 NFA: 开始态经一条标 ε 的边直接到接受态 (不读字符即可接受空串)。
- 单字符 a 的 NFA: 开始态经一条标 a 的边到接受态 (只接受单字符串 a)。



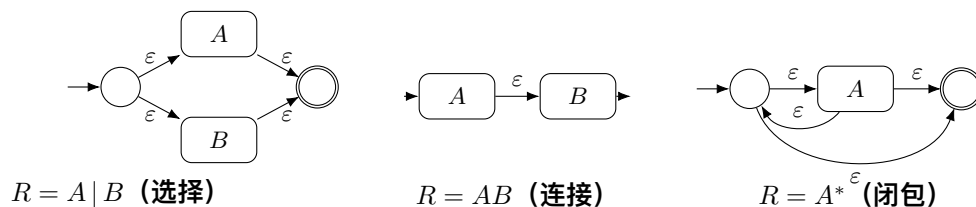
左: ε 的 NFA; 右: 单字符 a 的 NFA。

3.2 归纳: 三条复合规则 [p15]

设 RE A, B 已各自造好 NFA (每个都有唯一 start 和唯一 accept):

三条拼接规则 [p15]

1. **选择** $R = A|B$: 新建开始态, 用 ϵ 边分别进入 A, B 的开始态; A, B 的接受态各用 ϵ 边汇入新建的接受态。(“二选一”)
2. **连接** $R = AB$: 把 A 的接受态用 ϵ 边接到 B 的开始态; 整体开始态取 A 的、接受态取 B 的。(“先 A 后 B ”)
3. **闭包** $R = A^*$: 新建开始态, ϵ 边既指向 A 的开始态、也直达新接受态 (对应“零次”); A 的接受态用 ϵ 边回到 A 的开始态 (对应“多次”) 并指向新接受态。



例子: $(a|b)^*abb$ [p16,17]

分步搭积木: 先造原子 a, b 的 NFA $\rightarrow$ 用规则 得 $(a|b)$ $\rightarrow$ 用规则 得 $(a|b)^*$ $\rightarrow$ 用规则 把 $(a|b)^*$ 与 a, b, b 依次连接, 得到 $(a|b)^*abb$ 的完整 NFA。

3.3 回顾: 规范与实现如何衔接 [p18-22]

- **描述链 (Revisit)** [p18]: 语言规范 $\rightarrow$ 定义 **Token** $\rightarrow$ Token 的抽象是**模式 (Pattern)** $\rightarrow$ 模式由 RE 指定 (Specify); **词素 (Lexeme)** 是 Token 的实例 (*Instance*)、由模式**匹配 (Match)** 得到。
- **复合 RE 例子** [p20]: $(ab)^*, (a|b)^*, (a^*b^*)^*$ 。
- **消歧 (Removing Ambiguity)** [p20]: 关键字优先 (keyword first)、最长匹配 (maximal match)、先列者优先 (the one listed first)。
- **实现 = 有穷自动机** [p20]: NFA / DFA / 转换表; **Thompson 算法**把小 NFA 系统地连接成大 NFA。

4 子集构造法: $NFA \rightarrow DFA$ (第 22–31 页)

4.1 为什么要转成 DFA [p22,23]

等价定理 $L(NFA) \equiv L(DFA)$ [p23]

NFA 与 DFA **描述能力相同**: 二者都识别正则语言; 对每个 NFA M , 都存在一个 DFA M' 使 $L(M) = L(M')$ 。

各有取舍 [p23]

DFA **更费内存** (转移表可能更大), 但**执行更快**: DFA 的转移次数 = 输入长度; NFA 潜在转移数更多。所以实践中常做 $NFA \rightarrow DFA$ 转换——DFA 的速度优势远超它多占的内存。

4.2 核心思想与两个待解决问题 [p24,25]

子集构造法 (Subset Construction) [p24]

DFA 的每一个状态, 对应 NFA 的一个状态集合。

直觉: 读完输入 $a_1a_2 \cdots a_n$ 后, DFA 所处的那个状态, 正好对应“NFA 从开始态出发、沿 $a_1a_2 \cdots a_n$ 能到达的所有 NFA 状态的集合”。这样就把 NFA 的“同时在多个状态”用 DFA 的“一个集合状态”模拟掉。

要把 NFA 变 DFA, 必须消除两种“非确定性” [p25]:

1. 消除 ϵ 转移;
2. 消除同一状态对同一字符的多个转移。

4.3 两个关键运算: ϵ -closure 与 move [p25,26]

ϵ -闭包 (ϵ -closure) [p25,26]

ϵ -closure(s): 从状态 s 出发, 经过零次或多次 ϵ 转移能到达的所有状态的集合。

ϵ -closure(T): 把集合 T 里每个状态的 ϵ -闭包并起来。

通俗讲: 站在某些状态上, “免费” (不读字符) 能溜达到的所有地方, 连同自己。例: 若 s 经 ϵ 可达 s_1, s_2 , 则 ϵ -closure($\{s\}$) = $\{s, s_1, s_2\}$ 。

move 运算 [p26]

$\text{move}(T, a) = \{t \mid \exists s \in T, s \xrightarrow{a} t\}$: 从集合 T 中某个状态出发、读一个字符 a 能直接到达的 NFA 状态集合。

组合记号： $I_a = \varepsilon\text{-closure}(\text{move}(I, a))$ ——先走一个 a ，再做 ε -闭包，这正是 DFA 中“集合状态 I 读 a 之后到达的新集合状态”。

4.4 完整算法（三步） [p27–30]

1. **Step 1** 起点： $I = \varepsilon\text{-closure}(\text{开始态})$ ，作为 DFA 的第一个状态。
2. **Step 2** 迭代：对每个尚未处理的集合状态 I 、每个字符 $x \in \Sigma$ ，求 $I_x = \varepsilon\text{-closure}(\text{move}(I, x))$ ；遇到新集合就登记为新 DFA 状态，直到不再产生新集合。
3. **Step 3** 标记接受态：凡包含原 NFA 任一接受态的集合状态，标记为 DFA 的接受态。

贯穿例子的转移表 [p27–31]

以一个 NFA（开始态 0， $0 \xrightarrow{\varepsilon} 1$ 、 $0 \xrightarrow{\varepsilon} 3$ 、 $1 \xrightarrow{a} 2$ 、 $3 \xrightarrow{b} 4$ ，4 为接受态）为例。按算法逐步求出下表（ I_a, I_b 列即转移目标）：

DFA 状态	集合 I	I_a (读 a)	I_b (读 b)	接受?
T_0	$\{0, 1, 3\}$	$\{2, 4\} = T_1$	$\{4\} = T_2$	否
T_1	$\{2, 4\}$	$\{\}$ (死状态 dead)	$\{1, 3\} = T_3$	是
T_2	$\{4\}$	$\{\}$	$\{\}$	是
T_3	$\{1, 3\}$	$\{2, 4\} = T_1$	$\{4\} = T_2$	否

易错点：死状态 (dead state) [p29]

$\text{move}(\{2, 4\}, a) = \{\}$ (空集)，意味着读到 a 后无路可走——这对应一个死状态 (陷阱状态)。在严格的 DFA 里，所有“无定义的转移”都指向这个死状态；它不是接受态，进去就出不来、整串被拒。(教材 3.8.3)

5 DFA 最小化：分割法（第 32–40 页）

5.1 为什么能最小化、目标是什么 [p33]

最小 DFA 的唯一性 [p33]

对任意给定 DFA，都存在一个识别相同语言、状态数最少的等价 DFA，而且这个最小 DFA 是唯一的（在状态重命名意义下）。

等价状态 (Equivalent States) [p33]

两个状态 s, t 等价，当且仅当同时满足：

1. s, t 要么都是接受态、要么都是非接受态（同类）；
2. 对每个字符 $x \in \Sigma$ ， s 与 t 在 x 上转移到的状态也是等价的。

通俗讲：两个状态等价 = ”无论后面再输入什么串，都无法区分它们”（要么都接受、要么都拒）。

5.2 分割算法 (Partition Algorithm) [p34]

1. **初始**：把所有状态分成两组——{ 接受态 } 和 { 非接受态 }。
2. **细分**：对每个组、每个字符 x ，看组内各状态在 x 上是否转移到同一组；若不一致，就把这个组拆开。
3. **重复**直到没有任何组再能被拆分；每个最终组合成最小 DFA 的一个状态。

简单例子 [p35]

初始 $\{A\}$ （非接受）、 $\{BC, AC\}$ （接受）。检查 $\{BC, AC\}$ ：BC 与 AC 在 0 上都到 AC、在 1 上都到 BC——任何串都无法区分，故不拆。最终 $\{A\}$ 、 $\{BCAC\}$ 。

复杂例子 $\{S, A, B, C, D, E, F\}$ 的分割过程 [p36–39]

1. **初始**： $\{S, A, B\}$ （非接受）、 $\{C, D, E, F\}$ （接受）。
 2. 检查 $\{C, D, E, F\}$ ：对 a、b 转移都落在 $\{C, D, E, F\}$ 内部，**全等价，不拆**。
 3. 检查 $\{S, A, B\}$ ：读 a 时 $S \rightarrow A$ ， $B \rightarrow A$ 但 $A \rightarrow C$ （去了另一组） $\Rightarrow A$ 与 $\{S, B\}$ 不等价，拆出 $\{A\}$ 。
 4. 再检查 $\{S, B\}$ ：读 b 时 $S \rightarrow B$ ， $B \rightarrow \{C, \dots\}$ 不一致 $\Rightarrow$ 拆成 $\{S\}$ 、 $\{B\}$ 。
 5. **最终**： $\{C, D, E, F\}$ 、 $\{S\}$ 、 $\{A\}$ 、 $\{B\}$ ——把 $\{C, D, E, F\}$ 记作一个状态 C ，画出最小 DFA。
-

练习：判断是否已最小 [p40]

DFA (态 0,1,2,3)：初始 $\{0,3\}$ 、 $\{1,2\}$ 。检查 $\{1,2\}$ ：状态 2 无转移、状态 1 有 b 转移 $\Rightarrow$ 拆开。再查 $\{0,3\}$ ： $\text{move}(\{0,3\}, a) = \{1\}$ 、 $\text{move}(\{0,3\}, b) = \{2\}$ 一致 $\Rightarrow 0, 3$ 等价。结果 $\{0,3\}$ 、 $\{1\}$ 、 $\{2\}$ 。

6 复杂度分析 (第 41–42 页)

空间复杂度 [p41]

NFA 有 N 个状态, DFA 的每个状态是这 N 个状态的某个子集; 非空子集共 $2^N - 1$ 个, 故 DFA 状态数最坏 $O(2^N)$ 。

但好消息: 对真实的程序语言, NFA 与 DFA 的状态数往往差不多, 指数爆炸很少真正发生。

时间复杂度 [p42]

DFA 执行: $O(|X|)$ ($|X|$ = 输入长度)。每步常数时间: 当前态 S 、输入 c , 直接查 $T[S, c]$ 并更新当前态。

NFA 执行: $O(|X| \cdot N^2)$ 。每步 $O(N^2)$: 当前是”潜在状态集合”(最多 N 个), 读入 c 要对集合里每个态查转移并求并集 (最多 N 次、每次并 $O(N)$)。

确定 (DFA): 读 c 唯一转移 非确定 (NFA): 读 c 可去多个态



一句话权衡

DFA 用空间换速度: 建表时多花内存, 运行时一字一步、飞快; 这就是实际词法器几乎都选 DFA 的原因。

7 表驱动实现与 Lex (第 43–62 页)

7.1 Lex 的流水线 [p50]

Lex (词法分析器生成器) [p50]

Lex 自动把”正则规范”变成词法器, 流程为: $RE \rightarrow NFA \rightarrow DFA \rightarrow \text{最小化} \rightarrow \text{转移表} \rightarrow \text{表压缩}$ 。多数自动词法器都选 DFA 而非 NFA——以空间换速度。

- Lex 程序被转成转移表 + 动作 (actions), 交给一个 FA 模拟器运行。[p51]
- 自动机要能识别匹配程序中任一模式的词素。

7.2 多模式合并成一个 NFA [p52]

把多条规则拼起来 [p52]

三条规则 $a\{action1\}$ 、 $abb\{action2\}$ 、 $a*b+\{action3\}$ 各自造一个 NFA；再新增一个公共开始态 0，用 ε 转移连到三个 NFA 的开始态，合成一个大 NFA。（“还没读任何符号时，三条规则都可能命中”，所以起点要能“分身”到三者。）

7.3 模拟运行与“备份” [p53,54]

- NFA 模拟 [p53]：输入 aaba, $\varepsilon\text{-closure}(0) = \{0, 1, 3, 7\}$ ；读到第 4 个符号后状态集变空——回退 (back up) 到最近一次“含接受态”的位置，得 $a*b+$ 匹配 aab，执行 action3；随后剩下的 a 再匹配 a。即得 $(a*b+, aab), (a, a)$ 。
- DFA 词法器 [p54]：输入 abba, 状态序列 $0137 \rightarrow 247 \rightarrow 58 \rightarrow 68$ ；到末尾 a 时 68 无转移，而 68 本身是接受态、报告模式 abb。

7.4 该匹配多长？消歧规则 [p55]

最长匹配与先列优先 [p55]

- 最长匹配 (longest / maximal match)：总是取能匹配的最长前缀。例 aabbb...对 $a*b+$ ：一直读 b 直到遇到下一个 a，把“开头的 a 们 + 尽可能多的 b 们”作为词素。
- 同长则先列者优先 (listed first / 先出现优先)：abb 同时匹配模式 2 和模式 3，因模式 2 列在前面，判为模式 2 的词素。

7.5 关键字怎么匹配 [p56]

两种处理关键字的方法 [p56]

要识别 Identifiers: `letter(letter|digit)*` 与 Keywords: `if, then, else:`

- 方法 1：给每个关键字写 RE，并放在标识符 RE 之前使其优先。⇒ 状态机臃肿 (bloated)。
- 方法 2：用同一条 RE 识别关键字和标识符，再靠一张关键字表区分。⇒ 状态机精简 (streamlined)，但需额外查表。

通常方法 2 更高效。

7.6 Lex DFA 例子与二义性 [p57–62]

- DFA 的接受态用“它识别出的模式”来标号。状态集 $\{6, 8\}$ 既可接受 `abb` 又可接受 `a*b+`；因 `abb` 列在前， $\{6, 8\}$ 归 `abb`。[p58,59]
- 思考 [p60–62]：该 DFA 是否最小？8, 68, 58 是否真等价？——取决于 `abb` 是不是关键字、以及用方法 1 还是方法 2，从而影响初始分组与能否合并。

8 正则语言的极限与展望 (第 63–68 页)

一个非正则语言 [p63,64]

$\Sigma = \{a, b\}$, $L = \{a^n b a^n \mid n \geq 0\} = \{b, aba, aabaa, aaabaaa, \dots\}$ (一个 `b` 两边裹着相同个数的 `a`)。

它不能用正则表达式描述 (写 `a*ba*` 是错的，因为它不要求两边 `a` 数相等)。

为什么正则语言力不从心 [p64,65]

根本原因：FA 没有记忆、不会计数 (cannot count)。要保证“见 `b` 前的 `a` 数 = 见 `b` 后的 `a` 数”，必须记住前面数了多少个——而有穷状态做不到。

因此：括号匹配非正则、任何嵌套结构 (`if...if...else...else`) 非正则、 $\{a^n b^n \mid n \geq 1\}$ 非正则、 $\{\{\{\dots\}\}\}$ 非正则、C 语言整体非正则。

定位与展望 [p65]

正则语言足以描述单词级的东西 (标识符、字符串、注释)，但它是表达力最弱的常用形式语言。要分析带嵌套结构的语法 (如 C 的嵌套循环)，需要更强的工具——下一步进入上下文无关文法 (CFG, Context-Free Grammar)，由语法分析 (Parsing) 处理。

8.1 全讲转换流程小结 [p66,67]

1. RE $\rightarrow$ NFA: Thompson 算法 (归纳构造)。
2. NFA $\rightarrow$ DFA: 子集构造法 (Subset-Construction)。
3. DFA 最小化: 分割法 (Partition)。
4. 最小 DFA $\rightarrow$ 表驱动: 生成转移表，由 FA 模拟器运行 $\rightarrow$ 词法分析器 $\rightarrow$ 交给语法分析器。

进一步阅读 (龙书) [p68]: 精读 3.6–3.7、3.9.6 (有穷自动机与转换); 略读 3.9.1–3.9.5 (RE 直接到 DFA)。

9 练习与典型题 (第 69–80 页)

- **判别与写 RE [p69]**: 给定图是 NFA (因有 ε 转移、且对输入 1 有多个转移); 它的 RE 是 $0^*1^+2^*$ 。引申: 写出 $0^+1^+2^+$ 的 NFA。
- **概念判断 [p70]**: NFA 的行为总能被 DFA 模拟 $\Rightarrow$ always (因 $L(\text{NFA}) \equiv L(\text{DFA})$)。
- **补集 [p71]**: $\Sigma = \{a\}$, 求所给 NFA 接受语言的补集 (complement) $\Rightarrow$ 答案 ε 。
- **Thompson 构造 [p72–74]**: 把 $(a|b)^*abb(a|b)^*$ 转成 NFA (按 $(a|b) \rightarrow (a|b)^* \rightarrow abb \rightarrow$ 拼接)。
- **从 RE 到最小 FA 全流程 [p75–80]**: 对”不含子串 abb”的语言:
 1. 写 RE: $b^*(a|ab)^*$;
 2. Thompson 构造 NFA;
 3. 子集构造法得 DFA (列转移表);
 4. 分割法最小化, 得 $\{0, 2\}$ 、 $\{1\}$ 、 $\{3\}$ 。

考点速记

一页速查

- **两类 FA**: DFA 单态/无 ε /单路径; NFA 多态/有 ε /多路径, 有一条到接受态即接受。
- **五元组**: $(\Sigma, S, n, F, \delta)$; $L(\text{FA}) \equiv L(\text{RE})$, $L(\text{NFA}) \equiv L(\text{DFA})$ 。
- **四步**: $\text{RE} \xrightarrow{\text{Thompson}} \text{NFA} \xrightarrow{\text{子集构造}} \text{DFA} \xrightarrow{\text{分割法}} \text{最小 DFA} \rightarrow \text{表驱动}$ 。
- **Thompson 三规则**: $A|B$ (双 ε 分叉汇合)、 AB (ε 串接)、 A^* (ε 自环 + 直达)。
- **子集构造**: ε -closure + move; $I_a = \varepsilon\text{-closure}(\text{move}(I, a))$; 空集 = 死状态。
- **等价状态**: 同接受性 + 每字符转移到等价态; 最小 DFA 唯一。
- **复杂度**: 空间 $O(2^N)$; DFA 执行 $O(|X|)$, NFA 执行 $O(|X| \cdot N^2)$; DFA 空间换速度。
- **消歧**: 最长匹配 + 同长先列优先; 关键字优先; 关键字表法通常更优。
- **极限**: FA 不能计数 $\Rightarrow \{a^n b^n\}$ 、括号匹配、嵌套结构非正则 $\Rightarrow$ 需 CFG。